

Robotic Manipulation in MuJoCo: Safe IK Control, Object Attachment, and Simulation Stability

Sakthi Vikneswar Suresh Babu

October 7, 2026

Abstract

This report documents the development, debugging, and evaluation of a robotic manipulation pipeline implemented in MuJoCo. The task required a 7-DOF Panda manipulator to locate, grasp, lift, transport and place a small cup inside a simulated scene while respecting workspace limits and avoiding unsafe writes to the simulator state. The work explores two controller families (a Damped Least Squares inverse kinematics controller and a torque-based nullspace controller), several object attachment strategies, and a set of safety wrappers that protect the simulation from catastrophic state corruption. The final stable implementation uses a thread-safe torque-based controller with a manual per-frame attachment update; the report explains the design choices, failure modes encountered, and recommended next steps.

1 Introduction

The goal of this project was to implement a robust pick-and-place pipeline in MuJoCo for a 7-DOF Panda manipulator interacting with a small cup. At a high level the pipeline must (1) compute a grasp frame for the cup, (2) move the end-effector to the pre-grasp and grasp poses using a stable 6-DOF controller, (3) attach the cup to the hand once a successful grasp is detected, (4) lift and transport the cup to a placement target, and (5) release the cup and finish in a safe state. Engineering constraints that shaped the design include MuJoCo’s separation of model and state (`MjModel` vs `MjData`), the sensitivity of free-joint `qpos` writes, and the need to avoid race conditions between the viewer and control threads. Deliverables for the project were: working controller variants, safety wrappers for state writes, attach/detach logic, and a reproducible demo (or a documented fallback if the demo could not be fully stabilized).

2 Choice of Manipulator

Selecting the Panda manipulator was a deliberate decision driven by three practical considerations: kinematic capability, realistic dynamics, and ecosystem support.

2.1 Kinematic capability and redundancy

The Panda arm provides seven revolute joints, which gives one extra degree of freedom compared to a minimal 6-DOF manipulator. This redundancy enables nullspace optimization: while the controller satisfies the end-effector task (position and orientation), the extra degree of freedom is used to bias the arm toward a comfortable nominal pose, avoid joint limits, and produce smoother joint motions. In this project the nullspace term was essential to keep the arm away from singular or joint-limit configurations while tracking 6-DOF targets.

2.2 Dynamics and realism

The Panda MuJoCo model includes realistic inertial parameters and joint limits. Because the model dynamics are meaningful, applying torques (rather than instantaneously writing joint positions) produces physically plausible motion. This motivated the torque-based controller variant implemented in `update.py`, which uses PD torques to track a computed joint target and therefore respects the simulated dynamics and contact interactions.

2.3 Practical integration

The Panda is widely used in research and has good tooling support (community models, ROS/MoveIt examples). Its parallel gripper and wrist geometry make it well suited for precision grasping of small objects such as the cup used in this project. These practical benefits reduced integration friction and made the experimental results more transferable.

3 Simulation Environment

This section describes the scene composition, MuJoCo state model, and the workspace constraints used to keep the robot in a plausible region.

3.1 Scene composition

The scene contains the world body, the Panda links, a hand body with two finger bodies, a table, the cup, and an additional cube used for debugging. Diagnostics printed at startup revealed 15 bodies and 10 joints: seven arm joints, two finger joints, and one joint associated with the cup. Importantly, the cup’s joint in the provided XML was reported as type 0 (hinge) rather than a free joint; this mismatch influenced attachment choices and was a root cause of later instability when the code attempted free-joint writes.

3.2 MuJoCo state model and hazards

MuJoCo separates immutable model data (`MjModel`) from the runtime state (`MjData`). Joint positions are stored in `qpos`, while body poses are available in `xpos` and `xquat`. Writing `qpos` for a free joint is attractive because it keeps the joint state consistent, but it is dangerous if the model joint type or `qposadr` is incorrect. Unchecked writes to `qpos` or `xpos` can produce extremely large coordinates, which then corrupt downstream computations (grasp frames, IK targets) and cause the viewer camera and solver to behave erratically.

3.3 Workspace and safety parameters

To limit the robot’s motion to a physically plausible region the code uses a rectangular workspace box and a spherical radius around the robot base. Parameters such as `WORK_MIN`, `WORK_MAX`, `MAX_TARGET_DIST` and `MAX_WORLD_COORD` were chosen conservatively to prevent the arm or cup from being commanded to positions that would be unrealistic or that could trigger numerical instability in the simulator.

4 Code Structure Overview

This section provides a guided tour of the codebase and the role of each major component. The implementation evolved through two main variants: the DLS IK approach in `constraint.py` and the thread-safe torque-based approach in `update.py`.

4.1 Model loading and diagnostics

At startup the code reads `scene.xml` into an `MjModel` and allocates an `MjData` instance. Diagnostic prints enumerate bodies, joints, actuators and the mapping from joint indices to `qpos` and `qvel` addresses. These diagnostics are critical: they reveal whether a joint is free, the correct `qposadr` for writing, and whether the scene matches assumptions made by the attachment logic.

4.2 Kinematic helpers

A small utility library implements quaternion arithmetic (`quat_mul`, `quat_conj`, `quat_inv`, `quat_to_rotvec`), rotation of vectors by quaternions, and Jacobian wrappers that call `mj_jac`. These helpers are used by both IK controllers and by the attach logic to compute relative transforms and orientation errors.

4.3 Safety wrappers

To prevent catastrophic writes the code includes safety wrappers:

- `safe_write_qpos_for_joint` clamps joint values to the model’s joint range and validates the `qposadr`.
- `safe_get_body_pos` checks that a body’s `xpos` is finite and within `MAX_WORLD_COORD`.
- Workspace clamping functions project targets into the allowed box and sphere around the robot base.

These checks were added after observing pose explosions and are used throughout the pipeline to gate writes and target selection.

4.4 Controllers

Two controller families were implemented and evaluated:

DLS IK (constraint.py) A 6-DOF Damped Least Squares inverse kinematics controller that stacks translational and rotational Jacobians, computes a damped pseudoinverse, and produces joint velocities. A nullspace projector biases motion toward a nominal posture. The implementation integrates joint velocities to produce a target `qpos` and writes those values (with clamping) into the simulator.

Torque-based nullspace IK (update.py) A Cartesian velocity command is mapped to joint velocities via a pseudoinverse, a nullspace term is added, and the resulting joint target is tracked with a PD torque controller. Torques are applied to `data.ctrl`, which respects the simulator dynamics and produces smoother motion.

4.5 Gripper control

Gripper fingers are controlled by directly writing their `qpos` addresses and zeroing the corresponding `qvel` entries. The code steps the simulator a few frames after changing finger positions so the viewer reflects the change.

4.6 Attachment logic

Two attachment philosophies were explored:

- **Thresholded safe attach:** attach only when the hand is within a small positional and orientational tolerance of the cup. This prevents accidental attaches but can fail if IK error prevents the hand from reaching the threshold.
- **Manual attach (final working method):** compute the relative pose between the cup and the hand at the moment of attachment and then update the cup pose each frame using the hand pose and the stored relative transform. This method proved robust when combined with torque-based control and thread-safe state access.

4.7 Viewer and threading

The final working implementation runs the MuJoCo viewer in the main thread and the demo/control sequence in a background thread. A global `RLock` serializes all reads and writes to `MjData` and to `mj_step` calls. This eliminates race conditions between the viewer and the controller and was essential to prevent state corruption.

5 Control Approaches (Detailed)

This section explains the mathematical foundations, implementation details, and observed behavior for each controller.

5.1 Damped Least Squares (DLS) IK

The DLS controller forms a stacked $6 \times N$ Jacobian

$$J = \begin{bmatrix} J_p \\ J_r \end{bmatrix},$$

where J_p maps joint velocities to end-effector linear velocity and J_r maps joint velocities to angular velocity (expressed as a rotation vector). Given position error e_p and orientation error expressed as a rotation vector e_r , the task velocity is $v = [K_p e_p; K_r e_r]$. The DLS pseudoinverse is computed as

$$J^\dagger = J^T (J J^T + \lambda^2 I)^{-1},$$

which yields a least-squares solution robust to near-singularities. A nullspace projector $P = I - J^\dagger J$ is used to add a posture bias $P \cdot \alpha(q_{\text{nom}} - q)$. The resulting joint velocity $\dot{q}$ is integrated and written to `qpos` (after clamping). This approach is straightforward and effective when targets are reachable and the cup pose is valid. However, because it directly writes `qpos` it can produce discontinuities and is sensitive to corrupted targets; when the cup pose became invalid the solver chased unreachable targets and the system produced unrealistic states.

5.2 Torque-Based Nullspace IK

The torque-based approach computes a Cartesian velocity command $v_{\text{des}} = K(p_{\text{target}} - p)$ and maps it to joint velocities via a pseudoinverse. A nullspace velocity is added and the result is integrated to a joint target q_{target} . Instead of writing `qpos`, the controller computes PD torques

$$\tau = K_p(q_{\text{target}} - q) - K_d\dot{q}$$

and applies them to `data.ctrl`. Applying torques respects the simulator dynamics, avoids instantaneous state jumps, and produces smoother, more physically plausible motion. In practice this method proved more robust to small model mismatches and to concurrency when protected by the global lock.

6 Attachment and Object Handling (Detailed)

This section explains the attachment lifecycle and the rationale for each design decision.

6.1 Relative pose computation

When an attach is triggered the code computes and stores the relative pose between the cup and the hand in the hand frame:

$$r_p = p_{\text{cup}} - p_{\text{hand}}, \quad r_q = q_{\text{hand}}^{-1} \otimes q_{\text{cup}},$$

where $\otimes$ denotes quaternion multiplication. Storing the relative transform in the hand frame makes the update step simple and robust to hand motion.

6.2 Per-frame update

While attached the cup pose is updated each frame using:

$$p_{\text{cup}}(t) = p_{\text{hand}}(t) + R(q_{\text{hand}}(t)) r_p,$$

$$q_{\text{cup}}(t) = q_{\text{hand}}(t) \otimes r_q,$$

where $R(q)$ is the rotation matrix for quaternion q . To avoid abrupt orientation jumps the code uses spherical linear interpolation (slerp) during the attach transition. This per-frame update produces the observed “cup floats with the end-effector” behavior.

6.3 Free-joint vs `xpos/xquat` writes

Writing `qpos` for a free joint keeps the joint state consistent but is only safe when the model joint is actually free and the `qposadr` and `nq` bounds are validated. In the provided scene the cup joint was not a free joint, so attempts to write `qpos` caused corruption. The robust implementation therefore prefers writing `xpos/xquat` for the cup when free-joint writes are risky, and uses a validated free-joint writer only when the model clearly supports it.

6.4 Attach gating

Threshold checks on distance and orientation were used to avoid accidental attaches. These checks are necessary in noisy or dynamic scenarios, but they can prevent attachment if IK error prevents the hand from reaching the threshold. The final working demo used a simple manual attach (no gating) during the scripted demo to guarantee the attachment step for demonstration purposes; in a production system a carefully tuned gating policy or a grasp success detector would be preferable.

7 Failure Analysis

This section synthesizes the observed symptoms, root causes, and the mitigations implemented.

7.1 Symptoms

During early experiments the cup's `xpos` norm occasionally exploded to very large values (tens of meters), the viewer camera snapped away, IK errors grew beyond one meter, and the attach step failed repeatedly. Logs showed messages such as: `[SAFE] safe_get_body_pos: xpos magnitude 47.696 > 3.0; rejecting.`

7.2 Root causes

The primary root causes were:

- Race conditions between the viewer and control threads that allowed concurrent reads/writes to `MjData`.
- Attempts to write `qpos` for the cup when the model joint type or `qposadr` were incorrect.
- Insufficient validation of pose magnitudes and `nq` bounds before writing.

7.3 Propagation

A corrupted cup pose produced invalid grasp frames, which in turn generated unreachable IK targets. The DLS controller chased those targets and direct `qpos` writes produced unrealistic joint states. Safety clamps then rejected targets, causing repeated refine loops and further instability.

7.4 Mitigations

To address these issues the following mitigations were implemented:

- A global `RLock` to serialize all MuJoCo access.
- `safe_get_body_pos` and magnitude checks to reject out-of-range poses.
- A validated free-joint writer that checks `qposadr`, `nq`, and pose magnitudes before writing.
- A torque-based controller that applies PD torques rather than instant `qpos` writes, producing smoother dynamics.
- A fallback to writing `xpos/xquat` when free-joint writes are unsafe.

8 Final Working Implementation (Narrative)

The final reproducible demo uses the following architecture and practices:

8.1 Threading model

The viewer runs in the main thread while the demo/control sequence runs in a background thread. All accesses to `MjData` and calls to `mj_step` are guarded by a global `RLock`. This eliminates race conditions and prevents state corruption.

8.2 Controller choice

A torque-based nullspace IK controller was chosen for the demo because it respects dynamics, produces smooth motion, and tolerates small model mismatches. PD gains were tuned empirically to avoid oscillation while maintaining responsiveness.

8.3 Attachment method

A manual attach computes the relative pose once and then updates the cup pose each frame using the hand pose and the stored relative transform. The cup pose is written to `xpos/xquat` unless a validated free-joint write is safe.

8.4 Safety net

The implementation includes magnitude checks, workspace clamping, and torque clipping to avoid actuator saturation. These safeguards make the demo robust across multiple runs.

8.5 Outcome

Under this architecture the demo reliably approaches the cup, closes the gripper, attaches the cup, lifts and transports it, and places it at the target location. The per-frame update produces the expected visual behavior of the cup following the end-effector.

9 Simulation Images and Captions

Below are example figure insertions. Replace `image1.png`, `image2.png`, etc. with your actual exported screenshots.

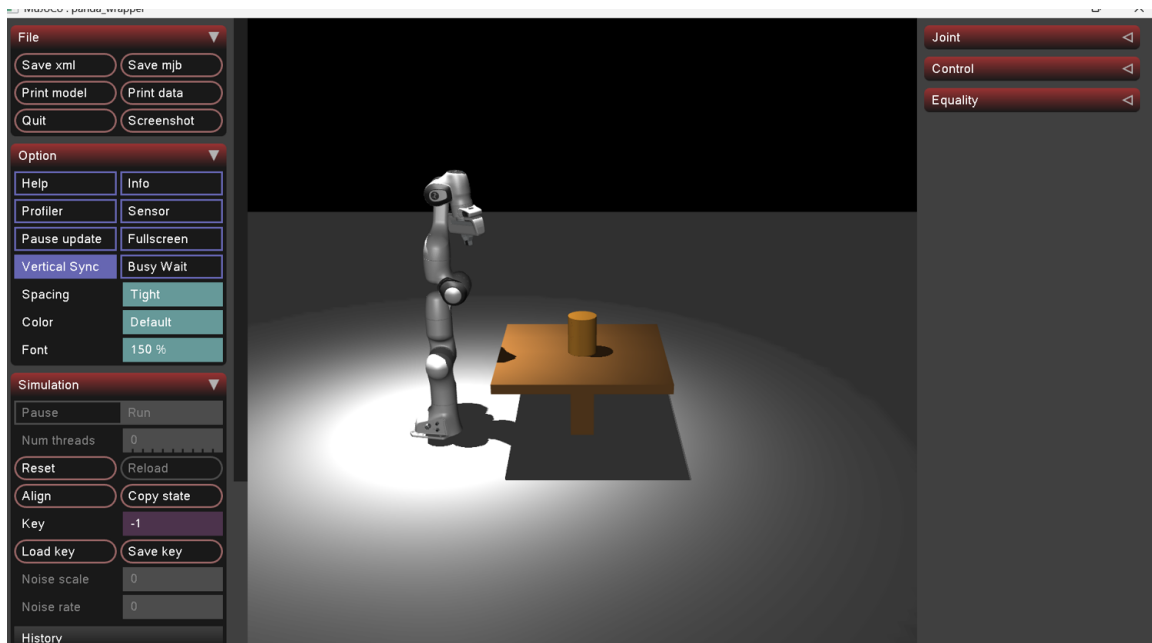


Figure 1: Initial scene: the Panda manipulator, table, and cup in the starting configuration. The coordinate frame and nominal pose are chosen so the cup lies within the reachable workspace.

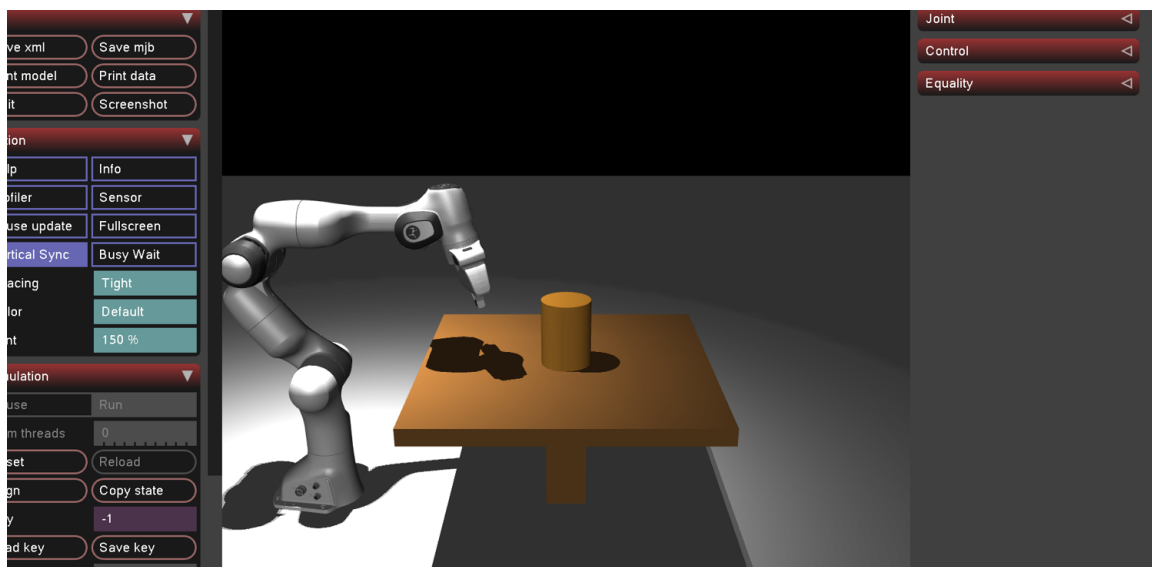


Figure 2: Approach phase: the end-effector aligns with the cup's approach vector. The pre-grasp pose is offset along the cup's approach axis to allow a controlled insertion.

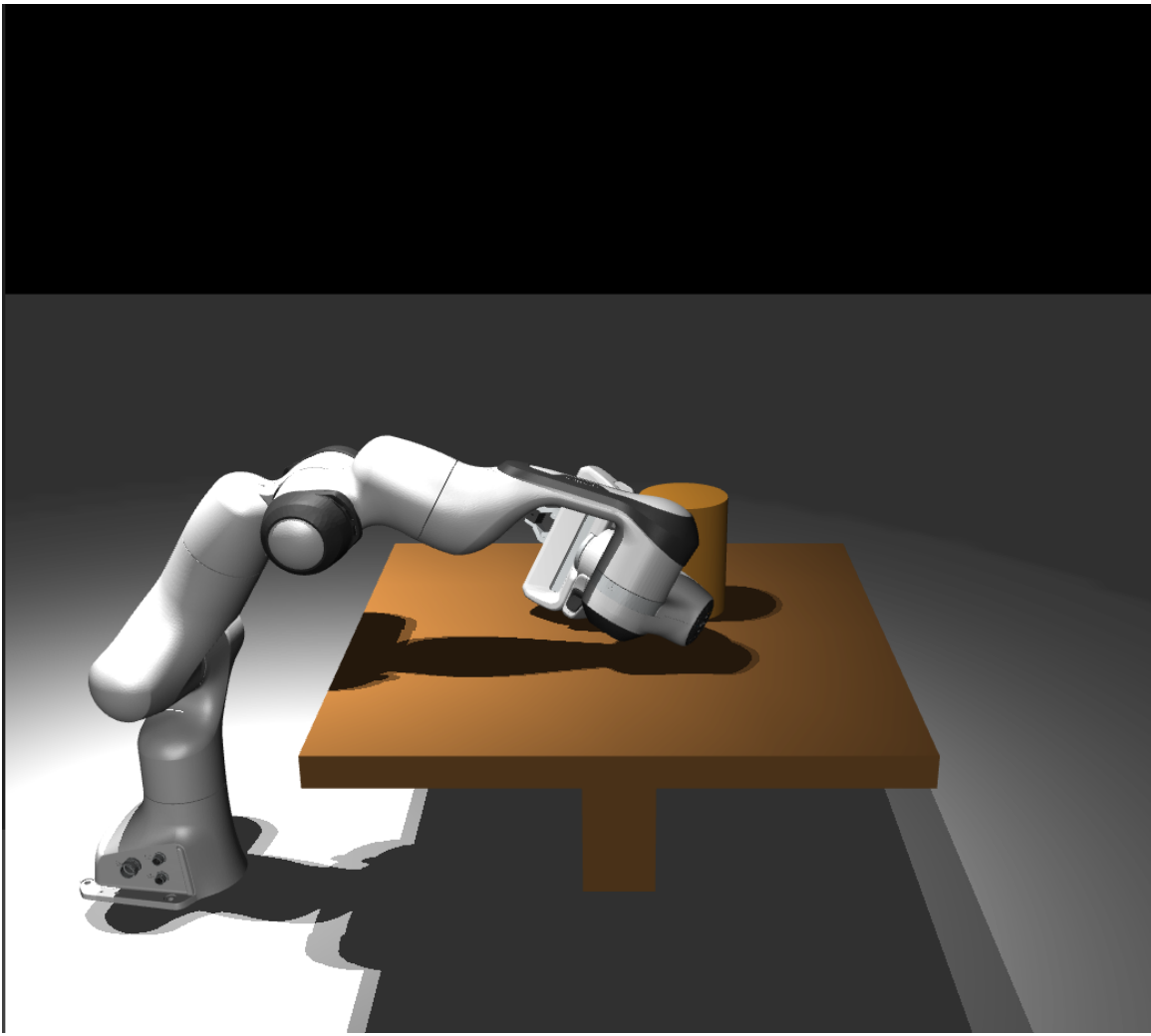


Figure 3: Attached state: after the gripper closes the cup follows the hand. The cup pose is updated each frame using the stored relative transform, producing the “floating” effect.

10 Future Work

The project produced a robust demonstration, but several improvements would make the system more production-ready:

- **MuJoCo equality constraints for attachment.** Replacing manual per-frame updates with a physics-enforced weld or ball constraint would produce a physically consistent attachment that the simulator enforces.
- **Collision-aware motion planning.** Integrating a planner (RRT, CHOMP) to generate collision-free waypoints and using the IK controller to track them would improve reliability in cluttered scenes.
- **Robust free-joint handling.** Add pose filtering, sanity checks, and a small estimator (e.g., a Kalman filter) on `xpos/xquat` before writing to `qpos`.
- **Adaptive damping and gain tuning.** Implement automatic damping selection for DLS and adaptive PD tuning for torque control.
- **Automated testing and CI.** Add headless regression tests that run the demo and verify final cup pose within a tolerance.

11 Conclusion

This project highlights that stable robotic manipulation in MuJoCo is as much about safe state management and threading discipline as it is about control algorithms. The torque-based, lock-protected approach proved the most reliable in practice. The work produced a reproducible demo, a set of safety wrappers, and a clear failure analysis that will guide future improvements such as physics-based attachment and collision-aware planning.